CS 4032 – Web Programming

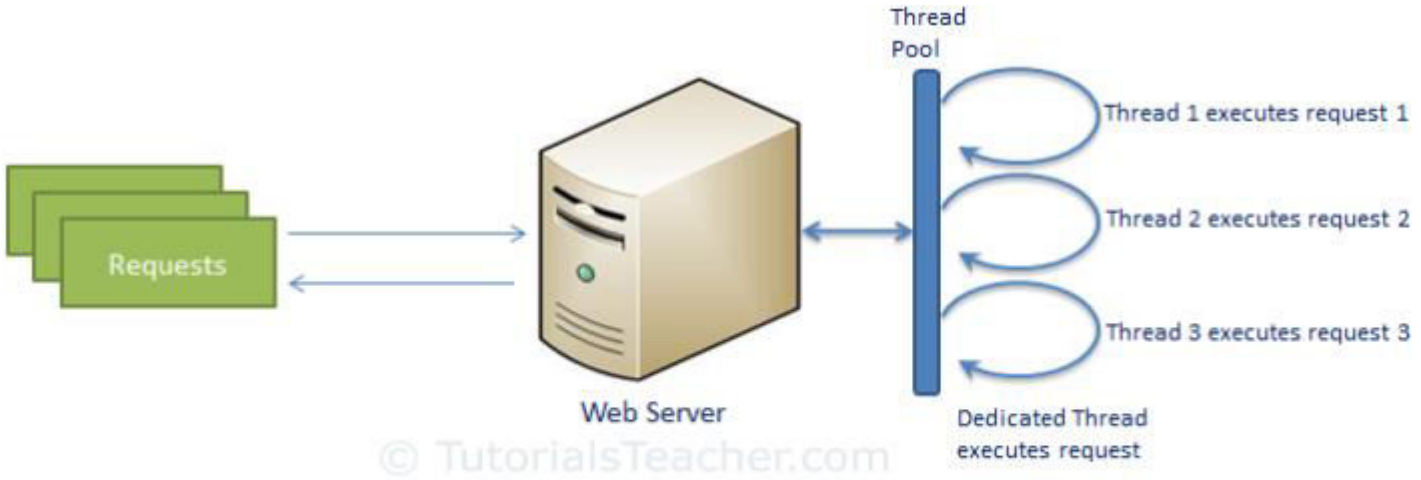
National University
Of Computer and Emerging Sciences


node
JS
express

What is node.js ?

- Created 2009
- Evented I/O for JavaScript
- Server Side JavaScript
- Runs on Google's V8 JavaScript Engine
- Support Asynchronous Programming

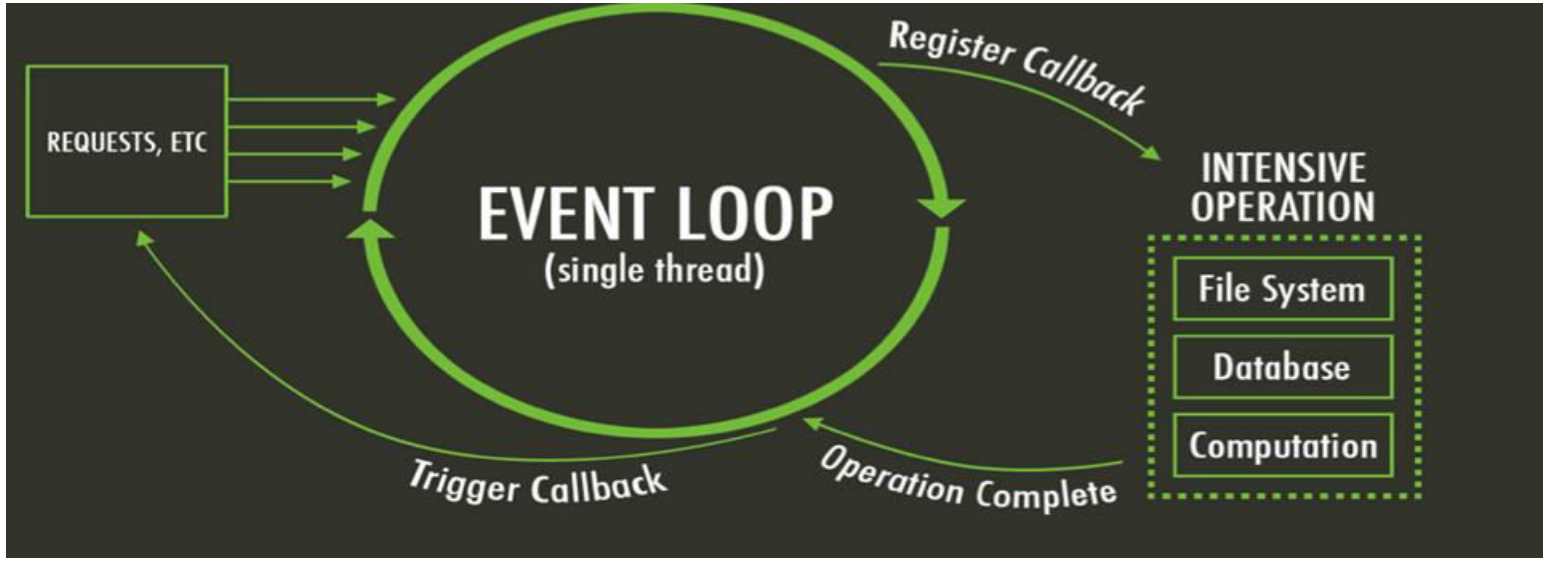
Traditional Web server



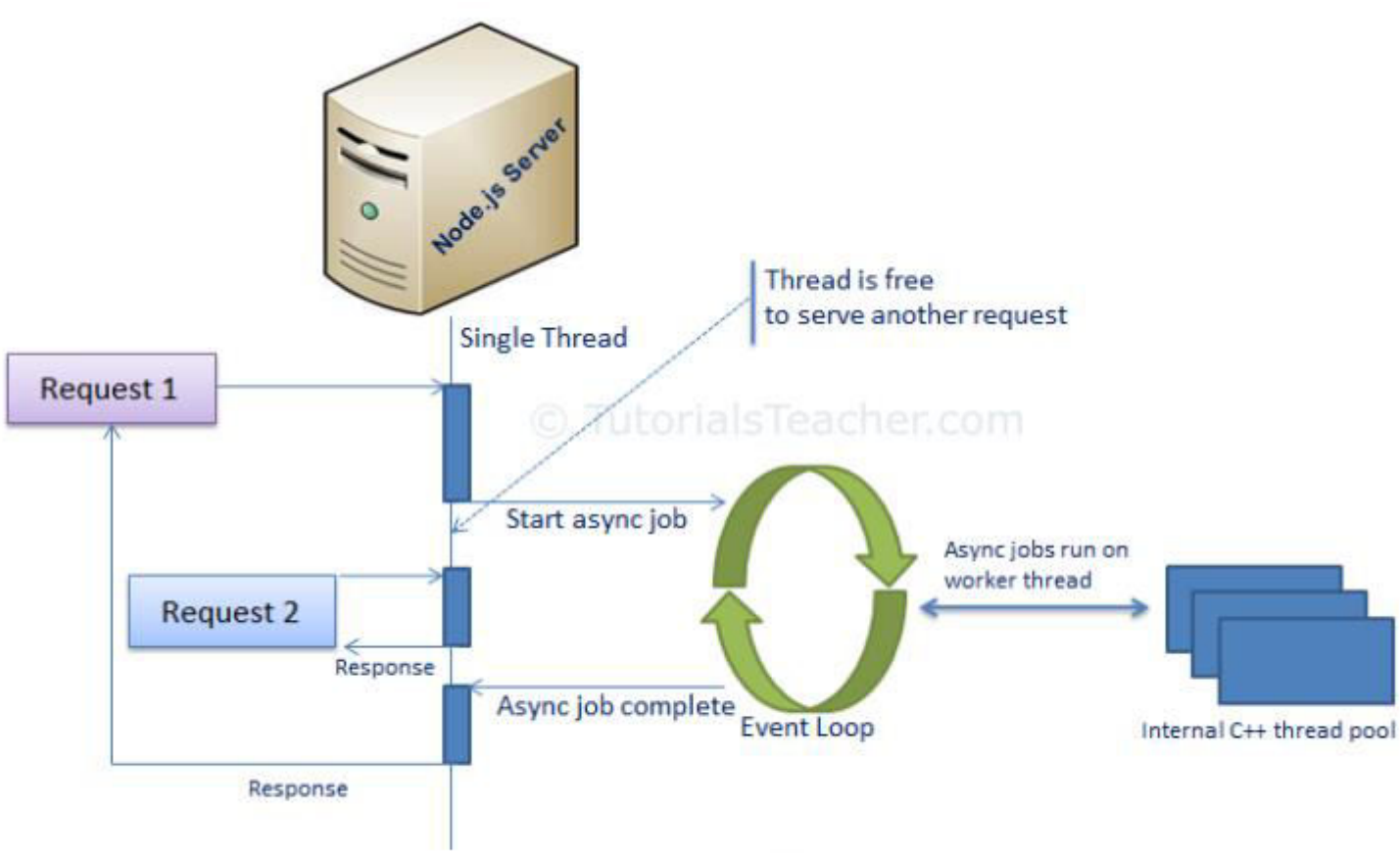
Traditional Web Server Model

What is unique about Node.js?

- JavaScript on server-side thus making communication between client and server will happen in same language
- Servers normally thread based but Node.JS is “Event” based. Node.JS serves each request in a Evented loop that can handle simultaneous requests.



Node.Js Server



Node.js Process Model

Why Use Node.js ?

- Node's goal is to provide an easy way to build scalable network programs.
- It lets you layered on top of the TCP library is a HTTP and HTTPS client/server.
- The JS executed by the V8 javascript engine (the thing that makes Google Chrome so fast)
- Node provides a JavaScript API to access the network and file system.

Standard JavaScript with

- Buffer
- C/C++ Addons
- Child Processes
- Cluster
- Console
- Crypto
- Debugger
- DNS
- Domain
- Events
- File System
- Globals
- HTTP
- HTTPS
- Modules
- Net
- OS
- Path
- Process
- Punycode
- Query Strings
- Readline
- REPL
- Stream
- String Decoder
- Timers
- TLS/SSL
- TTY
- UDP/Datagram
- URL
- Utilities
- VM
- ZLIB

... but without DOM manipulation

What can’t do with Node?

- Node is a platform for writing JavaScript applications outside web browsers. This is not the JavaScript we are familiar with in web browsers. There is no DOM built into Node, nor any other browser capability.
- Node can’t run on GUI, but run on terminal
- In the Node.js module system, each file is treated as a separate module.

Installing and using node Module

- Install a module…..inside your project directory
  - npm install <module name>
- Using module….. Inside your JavaScript code
  - var http = require(‘http’);
  - var fs = require(‘fs’);
  - var express = require(‘express’);

Hello World Example

- STEP 1: create directory and call npm install and follow instructions
  - >mkdir myapp
  - >cd myapp
- Use the npm init command to create a package.json file for your application. For more information, see Specifics of npm's package.json handling.
  - > npm init
- Prompts you for a number of things, such as the name and version of your application. For now, you can simply hit RETURN to accept the defaults

Hello World example (index.js)

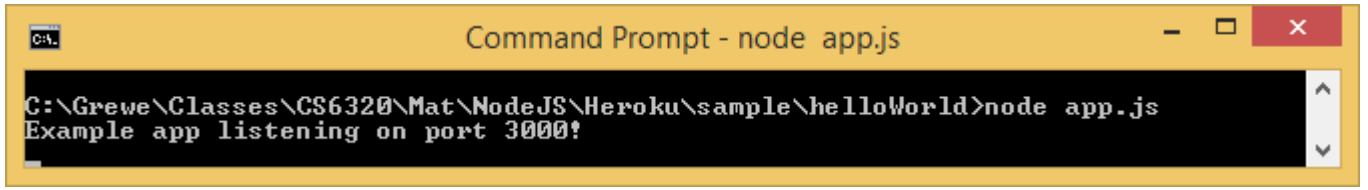
```
1. http.createServer(function (request, response) {
2.         // Send the HTTP header
3.         // HTTP Status: 200 : OK
4.         // Content Type: text/plain
5.          response.writeHead(200, {'Content-Type': 'text/plain'});
6.          // Send the response body as "Hello World“
7.          response.end('Hello World\n'); }).listen(8081);

8. // Console will print the message
9.  console.log('Server running at http://127.0.0.1:8081/');
```

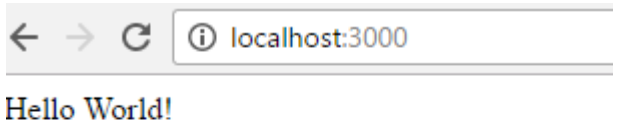
Hello World example –package.json – describes application

```
{
        "name": "helloworld",
        "version": "1.0.0",
        "description": "simple hello world app",
        "main": "index.js",
        "scripts": {
        "test": "echo \"Error: no test specified\" && exit 1"
        },
        "author": “XXX",
        "license": “XXX",
        "dependencies": {
        "express": "^X.XX.X"
        }
 }
```

Run your hello world application

- Run the app with the following command:
  - node index.js
- Then, load http://localhost:3000/ in a browser to see the output.





Asynchronous Programming

- Example : Web server
- Open a file on the server and return the content to the client

PHP, ASP, Others

1. Sends the task to the computer's file system.
2. Waits while the file system opens and reads the file.
3. Returns the content to the client.
4. Ready to handle the next request

Node

1. Sends the task to the computer's file system.
2. Ready to handle the next request.
3. When the file system has opened and read the file, the server returns the content to the client.

Asynchronous Programming …

```
callback(result) {
  // process the result
  // of task
  … …
}

some_task( callback ); /*
  calls to some_task()
  returns immediately
*/

// do other things
… …
```

… Asynchronous Programming

- Everything runs in one thread
- Asynchronous calls return immediately (a.k.a. *non-blocking*)
- A callback function is called when the result is ready

```
func(args…, callback(err,result))
```

- A.K.A. Event-driven Programming
  - A callback function is basically an event handler that handles the "result is ready" event

Asynchronous Programming Example 1

- `get_page.js`: request and print a web page
  - Use of the request package
  - Error-first callback style

```
"use strict";

const request1 = require("request");

function get_page_with_callback() {
  request1("http://nu.edu.pk", (err, response, body) => {
    console.log(body);
  });
}

get_page_with_callback();
```

Asynchronous Programming Example 2

- `write_file.js`: open a file, add one line, then close the file
  - Use of the File System package
    - open()
    - write()
    - close()
  - "Callback Hell" (a.k.a. "Pyramid of Doom")

```
"use strict";
const fs = require("fs");

fs.open("test.txt", "a", (err, fd) => {
 if (err) {
  console.log(err.message);
 } else {
  fs.write(fd, "A new line!\n", (err, bytesWritten, str) => {
   if (err) {
    console.log(err.message);
   } else {
    console.log(`${bytesWritten} bytes written.`);
   }
   fs.close(fd, err => {
    if (err) {
     console.log(err.message);
    }
   });
  });
 }
});
```

Promise

- A Promise is a JavaScript object
  - `executor`: a function that may take some time to complete. After it's finished, it sets the values of `state` and `result` based on whether the operation is successful
  - `state`: "pending" ➔ "fulfilled"/"rejected"
  - `result`: undefined ➔ value/error

```
var promise = doSomethingAync()
promise.then(onFulfilled, onRejected)
```

Use A Promise

```
promise.then(
  function(result) {/* handle result */},
  function(err) {/* handle error */ }
);
```

- After `executor` finishes, either the success handler or the error handler will be called and `result` will be passed as the argument to the handler

Other Common Usge of Promise

```
promise.then( success_handler );

promise.then( null, error_handler );

promise.catch( error_handler );

promise
  .then( success_handler )
  .catch( error_handler )
```

Promise Example

- `get_page_promise.js`: request and print a web page
  - Use of the request-promise-native package
  - `request()` returns a *promise*

```
const request2 = require("request-promise-native");

function get_page_with_promise() {
  request2("http://sun.calstatela.edu").then(body => console.log(body));
}
```

About Promise

- There can only be one result or an error
- Once a promise is settled, the result (or error) never changes
- `then()` can be called multiple times to register multiple handlers

Promises Chaining

- Suppose we have three functions `f1`, `f2`, `f3`
  - `f1` returns a Promise
  - `f2` relies on the result produced by `f1`
  - `f3` relies on the result produced by `f2`

```
f1.then(f2).then(f3)
```

Understand Promise Chaining …

`f1.then(f2)`

- then() returns a Promise based on the return value of the handler function
  - If `f2` return a regular value, the value becomes the result of the Promise
  - If `f2` return a promise, the result of that Promise becomes the result of the Promise returned by `then()`

… Understand Promising Chaining

```
f1.then(f2).then(f3)
```

- The result of `f1` is passed to `f2`
- The result of `f2` is passed to `f3`

Promise Example 2

- `write_file_promise.js`: open a file, add one line, then close the file
  - Use of promisify() in the Utilities package

Original function:

```
func(args…, callback(err,result))
```

Promisified:

```
func(args…) returns a Promise
```

```
"use strict";

const fs = require("fs");
const util = require("util");

const fopen = util.promisify(fs.open);
const fwrite = util.promisify(fs.write);
const fclose = util.promisify(fs.close);

let file = 0;

fopen("test.txt", "a")
  .then(fd => {
    file = fd;
    return fwrite(fd, "A New Line!\n");
  })
  .then(result => {
    console.log(`${result.bytesWritten} bytes written.`);
    return fclose(file);
  })
  .catch(err => console.log(err.message));
```

Parallel Execution

```
Promise.all([promise1, promise2 …]).then(
  function(results) {
    // results is an array of values, one
    // by each promise
  }
)

Promise.race([promise1, promise2 …]).then(
  function(result) {
    // result is the result of the promise
    // that settles first
  }
)
```

async and await

- `async` declares a function to be asynchronous
  - The return value of the function will be wrapped inside a Promise
- `await` waits until a Promise settles and returns its result
  - *`await` can only be used in an `async` function*

```
"use strict";

const fs = require("fs");
const util = require("util");

const fopen = util.promisify(fs.open);
const fwrite = util.promisify(fs.write);
const fclose = util.promisify(fs.close);

async function write_file() {
  try {
    let fd = await fopen("test.txt", "a");
    let result = await fwrite(fd, "A New Line!\n");
    console.log(`${result.bytesWritten} bytes written.`);
    await fclose(fd);
  } catch (err) {
    console.log(err);
  }
}

write_file();

// Because async function returns a promise, an alternative to try/catch
// is to handle the error in the error handler of the returned promise.
// For example: write_file().catch( err=>console.log(err) );
```

Running Node.js Server Applications

- Run server applications using nodemon during development
  - Automatically restart the application when changes in the project are detected
- Deploy server applications using pm2
  - Run server applications as managed background processes

Graceful Shutdown

- Properly save data and release resources (e.g. database connections) when a server application is stopped

The Process Object …

- process is a Node.js global object that provides information and control over the current Node.js process
- The beforeExit and exit event are not what you think – they are related to "normal" exit like when no more events are left in the event loop
- Instead, we need to handle Signal Events

… The Process Object …

- Process termination is a bit complicated
  - There are many ways to terminate a process
  - Some "termination" events do not accept event handlers
  - Different platforms work differently
  - Process managers like nodemon can further complicate things

… The Process Object

- Signal Events that should be handled for process termination
  - `SIGINT`: Ctrl-C, PM2 stop/reload/restart
  - `SIGTERM`: used by some cloud service like Heroku
  - `SIGUSR2`: nodemon restart

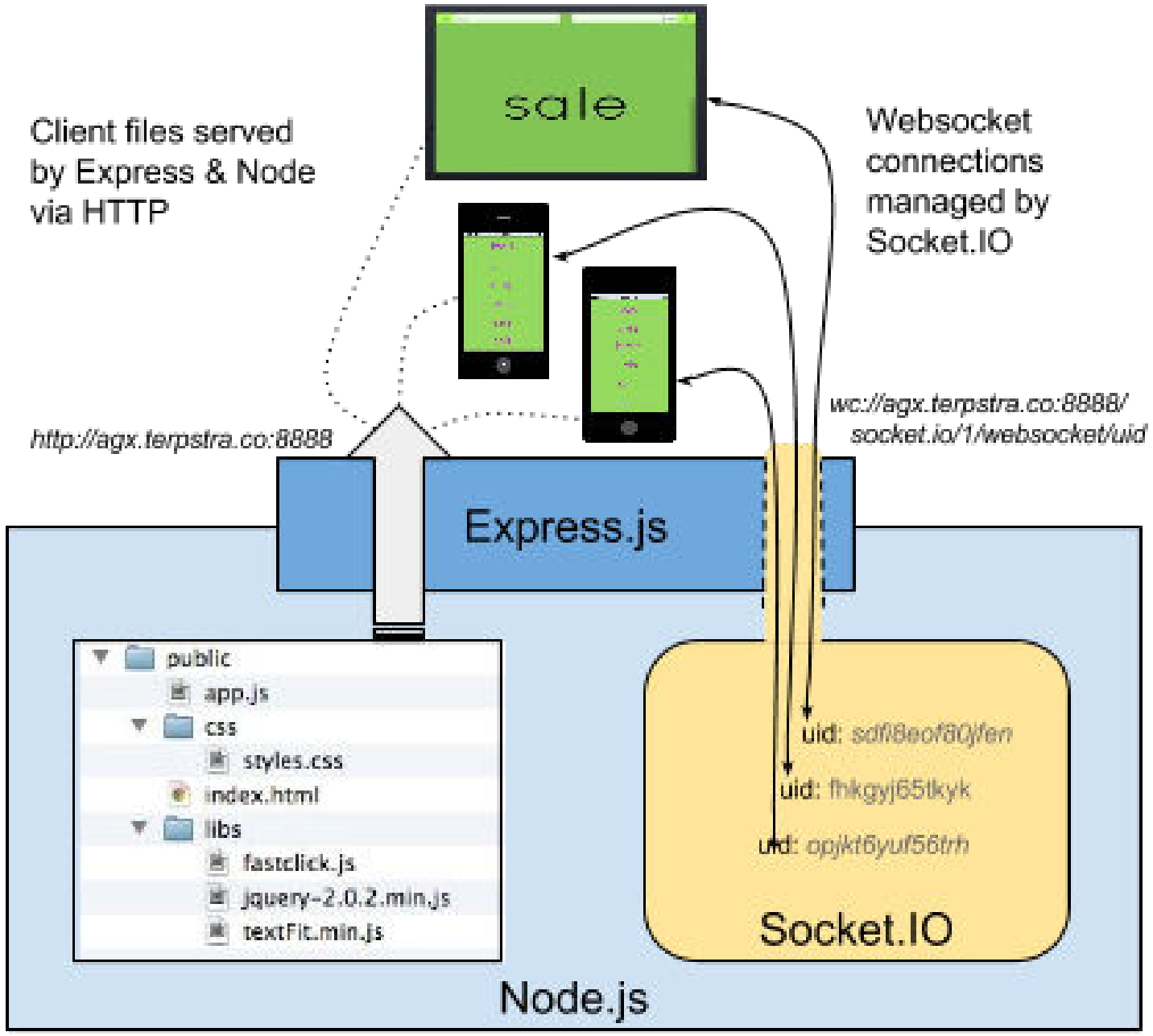
```
async function shutdown(signal, callback) {
  console.log(`${signal} received.`);
  await mongoose.disconnect();
  if (typeof callback === 'function') callback();
  else process.exit(0);
}

process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);
process.once('SIGUSR2', signal => {
  shutdown(signal, () => process.kill(process.pid, 'SIGUSR2'));
});
```

Example: webapp.js

```
const http = require("http");

const server = http.createServer();
server.on("request", (request, response) => {
  response.writeHead(200, { "Content-Type": "text/plain" });
  response.write("Hello world!");
  response.end();
});
server.listen(8080);

process.on("SIGINT", () => {
  console.log("SIGINT received. Cleanup before exit.");
  process.exit(0);
});

process.on("SIGTERM", () => {
  console.log("SIGTERM received. Cleanup before exit.");
  process.exit(0);
});

// . Nodemon uses SIGUSR2 to inform the program that it's about to be restarted.
// . process.kill() sends a signal to a process, not "killing" the process as
//   suggested by the name.
process.once("SIGUSR2", () => {
  console.log("SIGUSR2 received. Cleanup before restart.");
  process.kill(process.pid, "SIGUSR2");
});
```

Express

- Minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.



Express gives ease of functionality

- Routing
- Delivery of Static Files
- “Middleware” – some ease in development (functionality)
- Form Processing
- Simple forms of Authentication
- View support
- Basic error handling, e.g. rejecting malformed requests

A lot of this you can do in NodeJS
but, you may write more code to do it than
if you use the framework Express.

There are other alternatives than Express (the E in MEAN)
like Sail, Meteor

Express Basics

- Application
- Routing
- Handling requests
- Generating response
- Middleware
- Error handling

Express Installation

Assuming you've already installed Node.js, create a directory to hold your application, and make that your working directory.

```
$ mkdir myapp
$ cd myapp
```

Use the `npm init` command to create a `package.json` file for your application. For more information on how `package.json` works, see Specifics of npm's package.json handling.

```
$ npm init
```

This command prompts you for a number of things, such as the name and version of your application. For now, you can simply hit RETURN to accept the defaults for most of them, with the following exception:

```
entry point: (index.js)
```

Enter `app.js`, or whatever you want the name of the main file to be. If you want it to be `index.js`, hit RETURN to accept the suggested default file name.

Now install Express in the `myapp` directory and save it in the dependencies list. For example:

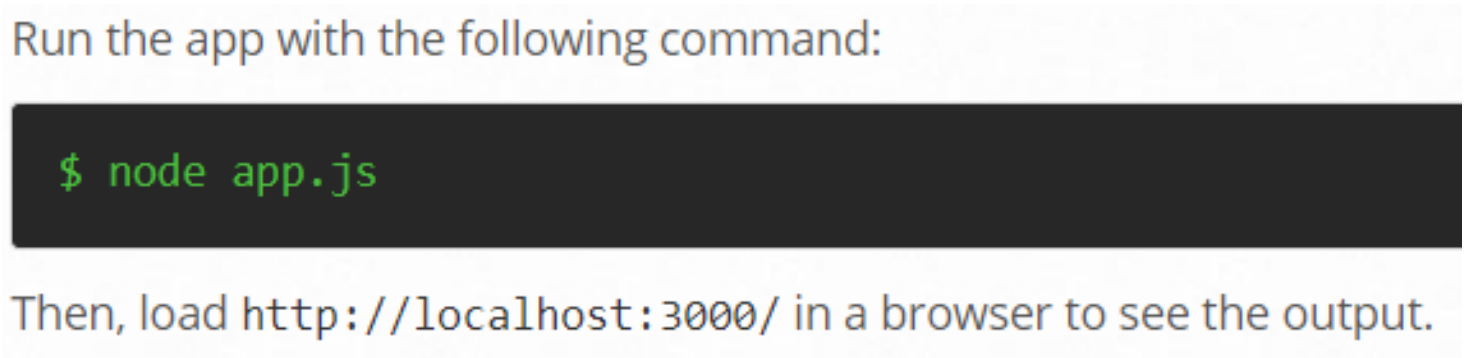
```
$ npm install express --save
```

To install Express temporarily and not add it to the dependencies list:

```
$ npm install express --no-save
```

HelloWorld in Express

```
const express = require('express');
const app = express();

app.get('/', (req, res) =>
  res.send('Hello World!'));

app.listen(3000, () =>
  console.log('Listening on port 3000'));
```



Application

```
const app = express();
```

- The Application object
  - Routing requests
  - Rendering views
  - Configuring middleware

Routing Methods in App

- `app.all( path, callback [,callback …])`

```
app.all('/secret', function (req, res, next) {
  console.log('Accessing the secret section ...')
  next() // pass control to the next handler
})
```

```
app.all('*', requireAuthentication, loadUser)
```

```
app.all('/api/*', requireAuthentication)
```

- `app.METHOD( path, callback, [,callback …])`
  - `METHOD` is one of the routing methods, e.g. `get`, `post`, and so on

```
app.get('/', function (req, res) {
  res.send('Hello World!')
})
```

Respond to POST request on the root route (/), the application's home page:

```
app.post('/', function (req, res) {
  res.send('Got a POST request')
})
```

Respond to a PUT request to the /user route:

```
app.put('/user', function (req, res) {
  res.send('Got a PUT request at /user')
})
```

Respond to a DELETE request to the /user route:

```
app.delete('/user', function (req, res) {
  res.send('Got a DELETE request at /user')
})
```

Modularize Endpoints Using Express Router …

- Example
  - List users: `/users/`, `GET`
  - Add user: `/users/`, `POST`
  - Get user: `/users/:id`, `GET`
  - Delete user: `/users/:id`, `DELETE`

… Modularize Endpoints Using Express Router

```
const router = express.Router();

router.get('/', …);
router.post('/', …);
… …

module.exports = router;
```

- A router is like a "mini app"

users.js

```
1  var express = require('express');
2  var router = express.Router();
3
4  const User = require('../models/user');
5
6  /* GET users listing. */
7  router.get('/', function(req, res, next) {
8    User.find( (err, users) => {
9       res.render('users', {title: 'Users', users: users});
10   });
11 });
12
13 module.exports = router;
```

… Modularize Endpoints Using Express Router

```
const users = require('./users');
app.use('/users', users);
```

- Attach the router to the main app at the URL

app.js

```
app.use('/', indexController);
app.use('/users', usersController);
app.use('/api/login', loginRestController);
```

Handling Requests

- Request
  - Properties for basic request information such as URL, method, cookies
  - Get header: get()
  - User input
    - Request parameters: req.query
    - Route parameters: req.params
    - Form data: req.body
    - JSON data: req.body

Example: Add

- GET: `/add?a=10&b=20`
- GET: `/add/a/10/b/20`
- POST (Form): `/add`
  - Body: `a=10&b=20`
- POST (JSON): `/add`
  - Content-Type: `application/json`
  - Body: `{"a": 10, "b": 20}`

```
Route path: /users/:userId/books/:bookId
Request URL: http://localhost:3000/users/34/books/8989
req.params: { "userId": "34", "bookId": "8989" }
```

```
app.get('/users/:userId/books/:bookId', function (req, res) {
  res.send(req.params)
})
```

Generating Response

- Response
  - Set status: status()
    - end()
  - Send JSON: json()
  - Send other data: send()
  - Redirect: redirect()
  - Other methods for set headers, cookies, download files etc.

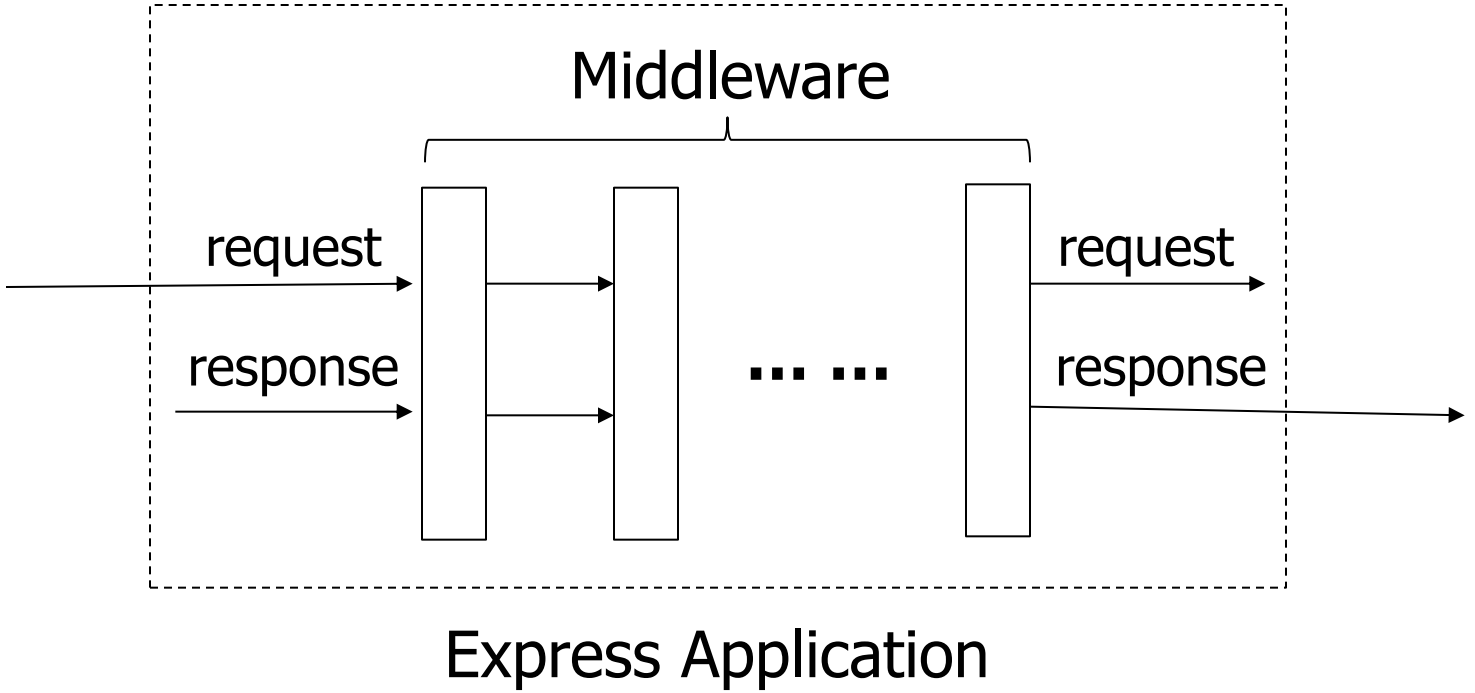
```
res
  .status(201)
  .cookie('access_token', 'Bearer ' + token, {
    expires: new Date(Date.now() + 8 * 3600000) // cookie will be removed after 8 hours
  })
  .cookie('test', 'test')
  .redirect(301, '/admin')
```

```
res.json(null)
res.json({ user: 'tobi' })
res.status(500).json({ error: 'message' })
```

```
app.get('/', function (req, res) {
  console.dir(res.headersSent) // false
  res.send('OK')
  console.dir(res.headersSent) // true
})
```

```
res.redirect('/foo/bar')
res.redirect('http://example.com')
res.redirect(301, 'http://example.com')
res.redirect('../login')
```

Middleware



- A middleware is a function that has access to three arguments: the `request` object, the `response` object, and a `next` function that passes control to the next middleware function

Middleware Example

- Create a simple request logger middleware that prints out request URL, method, and time
  - The `next` argument
  - Add the middleware to the application using `app.use()`
  - Middleware can also be added at the router level with `router.use()`

```
var express = require('express')
var app = express()
var router = express.Router()

// simple logger for this router's requests
// all requests to this router will first hit this middleware
router.use(function (req, res, next) {
  console.log('%s %s %s', req.method, req.url, req.path)
  next()
})

// this will only be invoked if the path starts with /bar from the mount point
router.use('/bar', function (req, res, next) {
  // ... maybe some additional /bar logging ...
  next()
})

// always invoked
router.use(function (req, res, next) {
  res.send('Hello World')
})

app.use('/foo', router)

app.listen(3000)
```

Other Middleware We've Seen So Far

- `express.json()` parses JSON request body and add JSON object properties to `req.body`
- `express.urlencoded()` parses urlencoded request body and request parameters to `req.body`
- Route handler functions are also middleware
  - Where is `next`??
  - Remember to use `next` if you have more than one handler functions for a route
- *Middleware order is important!*

Error Handling Middleware

- Error handling middleware has an extra argument `err`, e.g. `(err, req, res, next)`
- Calling `next(err)` will bypass the rest of the regular middleware and pass control to the next error handling middleware
  - `err` is an Error object
- Express adds a default error handling middleware at the end of the middleware chain

Build A MVC Web Application

- Understand the application structure created by Express Generator
- Data models and database access
- Controllers and views

```
$ npm install -g express-generator
$ express
```

```
$ npx express-generator
```

Application Structure

- `/public` for static resources
- `/routes` for controllers
- `/views` for view templates
- `/bin` for executables
- `package.json`
  - Packages
  - npm start

Customize Configuration Using Environment Variables

```
process.env.PORT || '3000'
```

- We often need to change runtime configuration such as port number, database url/username/password, log file location and so on
- Java applications usually use property files
- Node.js application prefer environment variables

Set Environment Variables Using `dotenv` Package

- Put the variables in a `.env` file, e.g.

```
DBURL=mongodb://localhost/webtest
USERNAME=cysun
PASSWORD=abcd
```

- Run `require('dotenv').config()` at the beginning of the application
- Include and version control a `.env.sample` file

Model and DB

- Add Mongoose models
- Connect to database
  - `mongoose.connect(<DBURL>)`
  - Mongoose automatically creates a connection pool
- Disconnect during shutdown

```
var Promise = require('promise');
var MongoClient = require('mongodb').MongoClient;
var url = 'mongodb://localhost/EmployeeDB';

MongoClient.connect(url)
    .then(function(err, db) {
        db.collection('Employee').updateOne({
            "EmployeeName": "Martin"
        }, {
            $set: {
                "EmployeeName": "Mohan"
            }
        });
    });
```

Handlebars Views

- `layout.hbs` is the default master page
  - Set a `layout` local to use a different master page
- A child view is combined with the master page as `{{{body}}}`
  - Triple braces mean do not escape content

```
<table>
  <tr><th>First Name</th><th>Last Name</th><th>Email</th></tr>
  {{#each users}}
  <tr>
      <td>{{firstName}}</td>
      <td>{{lastName}}</td>
      <td>{{email}}</td>
  </tr>
  {{/each}}
</table>
```

Implement REST API

- Mostly it's just MVC without View
- Authentication and authorization with JWT

Login Endpoint

- Use jsonwebtoken to create JWT

```
jwt = require('jsonwebtoken');
jwt.sign( <payload>, <secret> [, options] );
```

Usually a User object

Common claims such as `sub` and `exp` are in options

JWT Authorization Using Middleware

- Extract JWT token from Authorization header
- Verify the signature
- Attach the payload (i.e. the user object) to `req` and pass it onto the next middleware
- Return 401 if anything is wrong

```
var express = require('express');
var router = express.Router();
const User = require('../models/user');

const jwt = require('jsonwebtoken');
const jwtSecret = process.env.JWT_SECRET;

router.post('/', (req, res, next) => {
  User.findOne({
    firstName: req.body.username
  }, (err, user) => {
    if (err) return next(err);
    res.json({
      token: jwt.sign({
        name: user.firstName
      }, jwtSecret)
    });
  });
});

module.exports = router;
```

Passport

- Passport is a popular Node.js authentication framework
- Different authentication schemes (called *strategies*) can be implemented on top of Passport

Using Passport-JWT …

- Example: `passport-jwt.js`
- Registers JWT strategy with Passport
  - Extract JWT from Authorization header as a bearer token (other extractions also possible)
  - Decode and verify token
  - Passes decoded payload and a `done()` function to the callback function

```
1  const passport = require('passport');
2  const passportJWT = require('passport-jwt');
3
4  passport.use(new passportJWT.Strategy({
5    secretOrKey: process.env.JWT_SECRET,
6    jwtFromRequest: passportJWT.ExtractJwt.fromAuthHeaderAsBearerToken()
7  }, function (payload, done) {
8    return done(null, payload);
9  }));
10
11 module.exports = passport;
```

… Using Passport-JWT …

- The callback function allows further processing of the payload, e.g. loading a full user object from database
- `done(err, user)` sets `req.user` which can be used by subsequent middleware

… Using Passport-JWT

- Add Passport to an Express application

```
app.use( passport.initialize() );

app.use( '/api/', passport.authenticate('jwt', {
    session: false,
    failWithError: true
}));
```

Readings

- Express Documentation

Thank You